📐 Structured Output Generation

Day 3 — Notebook 2 of 4 | Estimated Time: 30 minutes

What You'll Learn

- How to use Gemini's native JSON mode for guaranteed structured output
- Schema enforcement with `response_schema`
- Using Pydantic models to define output shapes
- Parsing and validating structured outputs

Setup

```
%pip install -U -q "google-genai>=1.0.0" pydantic
```

```
!pip install google-auth==2.47.0
```

```
Requirement already satisfied: google-auth==2.47.0 in /usr/local/lib/python3.12/dist-packages (2.47.0)
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from google-auth==2.47.0) (0.4.
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-packages (from google-auth==2.47.0) (4.9.1)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (from pyasn1-modules>=0.2.1->goog
```

```
import sys, os, json
import time

import json
import re
from google import genai
from google.genai import types
from IPython.display import Markdown


try:
    from google.colab import auth
    auth.authenticate_user()
    print("✅ Authenticated via Colab")
except ImportError:
    print("✅ Using local credentials (gcloud auth)")


client = genai.Client(
    vertexai=True,
    project="chanakya-476010",
    location="us-central1",
)
MODEL_ID = "gemini-2.5-flash"
print("✅ Ready!")
```

```
✅ Authenticated via Colab
✅ Ready!
```

---

1. Gemini JSON Mode

Gemini can be configured to **always return valid JSON**. This is much more reliable than asking for JSON in the prompt.

```
# JSON mode — guarantees valid JSON output
response = client.models.generate_content(
    model=MODEL_ID,
    contents="List the 5 largest planets in our solar system with their diameter in km.",
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
    ),
)
```

```
# This is guaranteed to be valid JSON
data = json.loads(response.text)
print(json.dumps(data, indent=2))
```

```
[
  {
    "name": "Jupiter",
    "diameter_km": "139820"
  },
  {
    "name": "Saturn",
    "diameter_km": "116460"
  },
  {
    "name": "Uranus",
    "diameter_km": "50724"
  },
  {
    "name": "Neptune",
    "diameter_km": "49244"
  },
  {
    "name": "Earth",
    "diameter_km": "12742"
  }
]
```

---

2. Schema Enforcement with `response_schema`

You can define the **exact shape** of the JSON output using a schema.

```
# Define the schema for a book review analysis
response = client.models.generate_content(
    model=MODEL_ID,
    contents="""Analyze this book review:
    'The Pragmatic Programmer is an essential read for any developer.
    The examples are practical and the advice is timeless. However,
    some chapters feel dated. Overall rating: 4.5/5.'""",
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema={
            "type": "object",
            "properties": {
                "book_title": {"type": "string"},
                "sentiment": {"type": "string", "enum": ["positive", "negative", "mixed"]},
                "rating": {"type": "number"},
                "pros": {"type": "array", "items": {"type": "string"}},
                "cons": {"type": "array", "items": {"type": "string"}},
                "recommended_for": {"type": "string"},
            },
            "required": ["book_title", "sentiment", "rating", "pros", "cons"],
        },
    ),
)

result = json.loads(response.text)
print(json.dumps(result, indent=2))
```

```
{
  "book_title": "The Pragmatic Programmer",
  "sentiment": "mixed",
  "rating": 4.5,
  "pros": [
    "practical examples",
    "timeless advice"
  ],
  "cons": [
    "some chapters feel dated"
  ],
  "recommended_for": "any developer"
}
```

---

3. Using Pydantic Models

Pydantic is a Python library for data validation. Gemini supports passing Pydantic models directly as schemas.

```
from pydantic import BaseModel
from typing import Optional

# Define output schema using Pydantic
class JobPosting(BaseModel):
    title: str
    company: str
    location: str
    salary_min: Optional[int] = None
    salary_max: Optional[int] = None
    required_skills: list[str]
    experience_years: int
    remote_friendly: bool


response = client.models.generate_content(
    model=MODEL_ID,
    contents="""Extract job details from this posting:

    Senior ML Engineer at Google, Mountain View, CA (Remote OK).
    $180k-$250k. Requires 5+ years experience with Python, TensorFlow,
    and distributed systems. Nice to have: Kubernetes, MLOps experience.""",
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema=JobPosting,
    ),
)

# Parse into a Pydantic object for type-safe access
job = JobPosting.model_validate_json(response.text)
print(f"Title: {job.title}")
print(f"Company: {job.company}")
print(f"Salary: ${job.salary_min:,} - ${job.salary_max:,}")
print(f"Skills: {', '.join(job.required_skills)}")
print(f"Remote: {job.remote_friendly}")
```

```
Title: Senior ML Engineer
Company: Google
Salary: $180,000 - $250,000
Skills: Python, TensorFlow, distributed systems, Kubernetes, MLOps experience
Remote: True
```

---

4. Enum Outputs for Classification

Use `enum` to restrict outputs to a predefined set of values.

```
import enum

class TicketPriority(enum.Enum):
    CRITICAL = "critical"
    HIGH = "high"
    MEDIUM = "medium"
    LOW = "low"

class SupportTicket(BaseModel):
    category: str
    priority: TicketPriority
    summary: str
    requires_escalation: bool


tickets = [
    "The entire production database is down and no one can access the system!",
    "Can you help me change my profile picture?",
    "Our team's shared drive is running low on storage space.",
]

for ticket_text in tickets:
    response = client.models.generate_content(
        model=MODEL_ID,
        contents=f"Classify this support ticket:\n{ticket_text}",
```

```
            config=types.GenerateContentConfig(
                response_mime_type="application/json",
                response_schema=SupportTicket,
            ),
        )
        ticket = SupportTicket.model_validate_json(response.text)


        print(f"Ticket: {ticket_text[:50]}...")
        print(f"  Priority: {ticket.priority.value} | Escalate: {ticket.requires_escalation}")
        print()
```

```
Ticket: The entire production database is down and no one ...
  Priority: critical | Escalate: True

Ticket: Can you help me change my profile picture?...
  Priority: low | Escalate: False

Ticket: Our team's shared drive is running low on storage ...
  Priority: medium | Escalate: False
```

🏋️ Exercise 1: Invoice Data Extractor

Define a Pydantic model for invoice data and use Gemini to extract it.

```
# Exercise 1: Invoice extractor
# TODO: Define a Pydantic model for an invoice

class InvoiceItem(BaseModel):
    description: str
    quantity: int
    unit_price: float
    # TODO: Add a total field
    total : float

class Invoice(BaseModel):
    # TODO: Define fields for: invoice_number, date, vendor, items, subtotal, tax, total
    invoice_number: str
    date: str
    vendor: str
    items: InvoiceItem
    subtotal : float
    tax: float
    total : float

invoice_text = """
INVOICE #INV-2024-0847
Date: March 15, 2024
From: TechSupply Corp

Items:
- 5x Wireless Mouse @ $25.00 each
- 3x USB-C Hub @ $45.00 each
- 10x HDMI Cable @ $12.00 each

Subtotal: $380.00
Tax (8%): $30.40
Total: $410.40
"""

# TODO: Use Gemini with your schema to extract the data

response = client.models.generate_content(
    model=MODEL_ID,
    contents=invoice_text,
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema=Invoice,
    ),
)

# Parse into a Pydantic object for type-safe access
job = Invoice.model_validate_json(response.text)
```

```
print(job)
```

```
invoice_number='INV-2024-0847' date='March 15, 2024' vendor='TechSupply Corp' items=InvoiceItem(description='Wireless Mouse', qu
```

🏋️ Exercise 2: Batch Processing with Structured Output

Process multiple customer reviews and output a summary table.

```
# Exercise 2: Batch review processing

class ReviewAnalysis(BaseModel):
    sentiment: str  # TODO: Use enum for positive/negative/mixed
    rating: int     # 1-5
    key_topic: str

reviews = [
    "Amazing product! Fast shipping and great quality. Will buy again.",
    "Decent for the price but the build quality could be better.",
    "Terrible experience. Product arrived broken and support was unhelpful.",
    "Works as described. Nothing special but gets the job done.",
]

# TODO: Process each review with structured output and print a summary table
# Expected output: a table with columns: Review (truncated), Sentiment, Rating, Topic

for review_text in reviews:
    response = client.models.generate_content(
        model=MODEL_ID,
        contents=f"Classify the review:\n{review_text}",
        config=types.GenerateContentConfig(
            response_mime_type="application/json",
            response_schema=ReviewAnalysis,
        ),
    )
    rt = ReviewAnalysis.model_validate_json(response.text)


    print(f"Ticket: {review_text[:50]}...")
    print(f"Review: {rt}")
    print(f"  Sentiment: {rt.sentiment} | Rating: {rt.rating} | Topic: {rt.key_topic}")
```

```
Ticket: Amazing product! Fast shipping and great quality. ...
Review: sentiment='positive' rating=5 key_topic='Product and Service'
  Sentiment: positive | Rating: 5       | Topic: Product and Service
Ticket: Decent for the price but the build quality could b...
Review: sentiment='Mixed' rating=3 key_topic='Build Quality'
  Sentiment: Mixed | Rating: 3  | Topic: Build Quality
Ticket: Terrible experience. Product arrived broken and su...
Review: sentiment='negative' rating=1 key_topic='Product defect'
  Sentiment: negative | Rating: 1       | Topic: Product defect
Ticket: Works as described. Nothing special but gets the j...
Review: sentiment='positive' rating=4 key_topic='functionality'
  Sentiment: positive | Rating: 4       | Topic: functionality
```

📚 Further Reading

Resource	Type	Link
Gemini Structured Output	Docs	ai.google.dev/gemini-api/docs/structured-output
Pydantic Documentation	Docs	docs.pydantic.dev
JSON Mode Best Practices	Blog	ai.google.dev/gemini-api/docs/json-mode

---

Next up: 03_ReAct_Prompting_Pattern.ipynb